

Automate Your Testing Workflow with Github Actions and Docker

Overview

In this lab, you'll configure a GitHub Actions pipeline to run your Playwright tests automatically. You'll also simulate how the same pipeline could scale into a Dockerized environment and eventually deploy to AWS or other cloud platforms.

Objective

Your objective is to automate test execution and reporting for every pull request and push to the main branch.

Tools used: Playwright

Scenario

You've built a robust Playwright test suite. But until it runs automatically on every code change, bugs can silently slip into production. Now it's time to integrate your suite into a real-world CI/CD workflow that ensures tests are continuously run, results are captured, and quality remains a shared responsibility.

Instructions for learners

Step 1 – Create a GitHub Actions Workflow

Inside your Playwright project, create a workflow file at

`.github/workflows/playwright.yml`

Use the following trigger configuration to ensure tests run on pull requests and main branch pushes:

```
on:  
  push:  
    branches: \[main]  
  pull\_request:  
    branches: \[main]
```

Step 2 – Define Your Job

Set up the environment by specifying the latest Ubuntu runner, checking out the repo, and installing the required tools and dependencies.

name: Playwright Test Suite

jobs:

test:

runs-on: ubuntu-latest

steps:

\- name: Checkout code

uses: actions/checkout@v3

```
...
```

```
name: Set up Node.js
uses: actions/setup-node@v3
with:
  node-version: 18
```

```
name: Install dependencies
run: npm ci
```

```
name: Install Playwright and Browsers
run: npx playwright install
```

```
name: Run Playwright Tests
run: npx playwright test --reporter=html
...
```

Step 3 – Upload Artifacts

After the tests run, upload HTML reports or traces so they can be viewed later. Add the following step after your test execution:

name: Upload Playwright Report
uses: actions/upload-artifact@v3
with:
name: playwright-report
path: playwright-report/

If you've configured traces or screenshots on failure in your Playwright config, those will also be captured and uploaded as part of the report.

Step 4 – Docker Integration (Optional Advanced Extension)**

If your test suite or app under test needs to run inside Docker, update your pipeline to build a Docker container first. A simplified Playwright + Node.js Dockerfile might look like this:

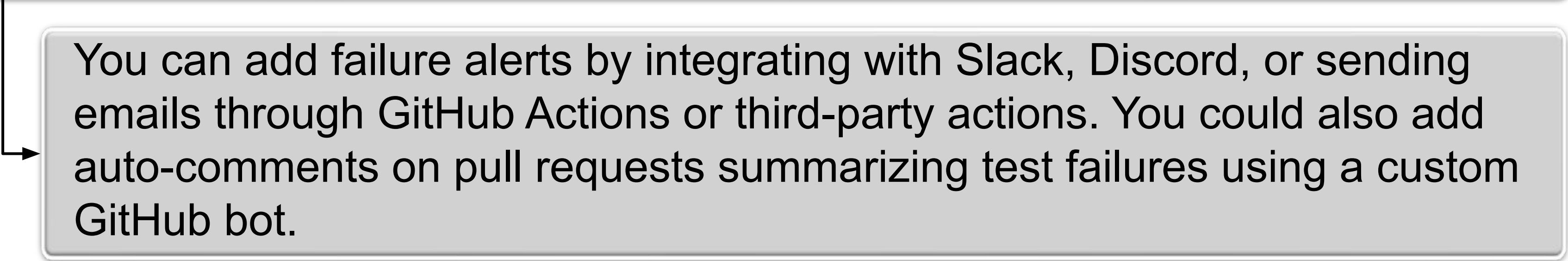
```
FROM mcr.microsoft.com/playwright:v1.41.0-focal
WORKDIR /app
COPY . .
RUN npm ci
ENTRYPOINT ["npx", "playwright", "test"]
```

Then update your GitHub workflow to build and use this image. Add the following before test execution:

- * name: Build Docker Image
run: docker build -t playwright-tests
- * name: Run Tests in Docker
run: docker run --rm playwright-tests

This approach isolates your test environment and mirrors a production-ready execution setup, which you could later extend to GitHub-hosted runners with AWS deployment targets.

Step 5 – Optional Alerting (Advanced)



You can add failure alerts by integrating with Slack, Discord, or sending emails through GitHub Actions or third-party actions. You could also add auto-comments on pull requests summarizing test failures using a custom GitHub bot.

Completion Criteria



Your GitHub Actions workflow should run automatically on every pull request and push to the main branch. The workflow must install dependencies, run tests, and upload at least one artifact such as an HTML report or trace. You should be able to demonstrate the workflow execution with a screenshot or GitHub run URL.

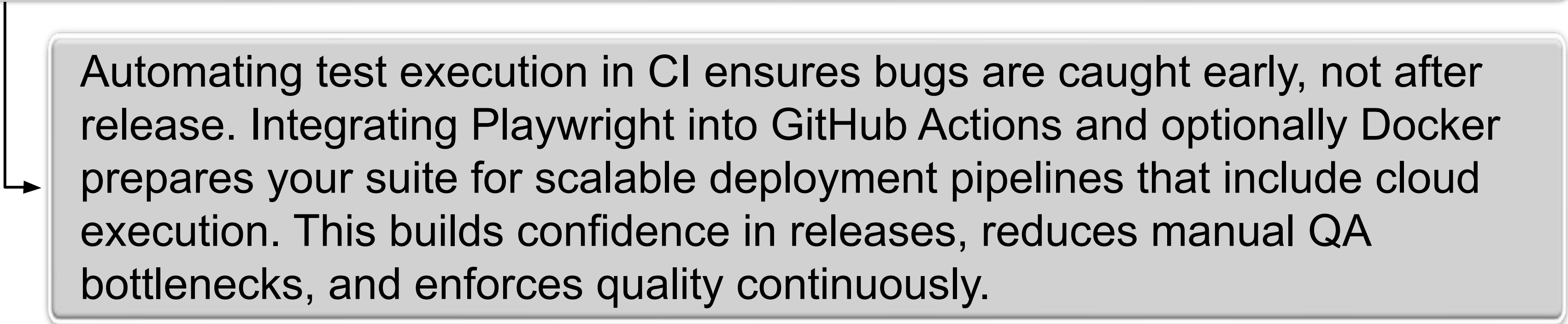
Reflection

Prompts

- Was your pipeline setup clean and repeatable
- Did your tests run reliably on every commit
- Were artifacts like HTML reports easy to access and review
- What changes would you make to scale this for teams or cloud runners
- What additional validations or alerts would make your CI more production-ready

Conclusion

Why This Matters



Automating test execution in CI ensures bugs are caught early, not after release. Integrating Playwright into GitHub Actions and optionally Docker prepares your suite for scalable deployment pipelines that include cloud execution. This builds confidence in releases, reduces manual QA bottlenecks, and enforces quality continuously.